LAB 06 - KẾT QUẢ THỰC HÀNH

Attention Mechanism | Bài 16-19

Bài 16 - Scaled dot-product attention

Hàm NumPy tính scores = QK^T / sqrt(d_k), attention_weights = softmax(scores) theo trục key, và context = attention_weights @ V; hàm trả về scores, attention_weights và context.

Tensor	Giá trị đầu vào
Q	[[0.5, -0.2]]
K	[[0.1, 0.4], [-0.3, 0.8], [0.7, 0.2]]
V	[[0.2, 0.5], [1.2, -0.1], [-0.4, 0.7]]

Đầu ra	Kết quả từ Colab
Scaled scores	[[-0.02121320, -0.21920310, 0.21920310]]
Attention weights	[[0.32339888, 0.26530937, 0.41129174]]
Context	[[0.21853433, 0.42307272]]
Tổng weights mỗi hàng	[1.0]

Bài 17 - Softmax ổn định số

Đại lượng	Kết quả từ Colab
Input scores	[1000.0, 1001.0, 1002.0]
Softmax	[0.09003057, 0.24472847, 0.66524096]
Tổng xác suất	0.9999999999999999 (xấp xỉ 1)

Câu hỏi: Vì sao trừ max(x) không làm đổi kết quả softmax?

Đặt m = max(x). Với mọi phần tử i:

softmax(x - m)_i = exp(x_i - m) / sum_j exp(x_j - m)
= [exp(-m) exp(x_i)] / [exp(-m) sum_j exp(x_j)]
= exp(x_i) / sum_j exp(x_j) = softmax(x)_i

exp(-m) là thừa số chung ở tử số và mẫu số nên triệt tiêu. Vì vậy phân phối softmax không đổi; phép dịch score xuống trước khi lấy exp giúp giảm nguy cơ overflow.

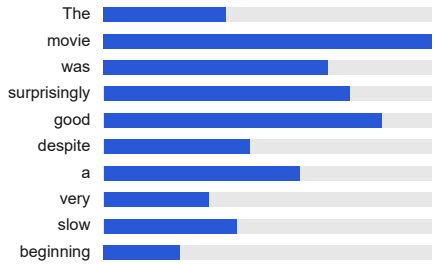
Nguồn output: notebook Google Colab LAB06_Bai16_17_Attention_NumPy, chạy bằng Python 3.

Bài 18 - Experiment 1: Attention distribution

Sequence: The movie was surprisingly good despite a very slow beginning.

Token	Score	Attention
The	-0.212991	0.066017
movie	0.773652	0.177070
was	0.393482	0.121071
surprisingly	0.484942	0.132666
good	0.607872	0.150020
despite	-0.039892	0.078493
a	0.256895	0.105614
very	-0.369226	0.056468
slow	-0.133643	0.071468
beginning	-0.686569	0.041113

Attention weights



Câu hỏi: Model đang phân phối trọng số như thế nào?

Trọng số được chia cho cả 10 token; cao nhất là “movie” (0.177070), tiếp đến “good” (0.150020), và thấp nhất là “beginning” (0.041113). Tổng các trọng số xấp xỉ 1.

Nguồn output: notebook Google Colab LAB06_Bai18_Attention_Distribution, chạy bằng Python 3.

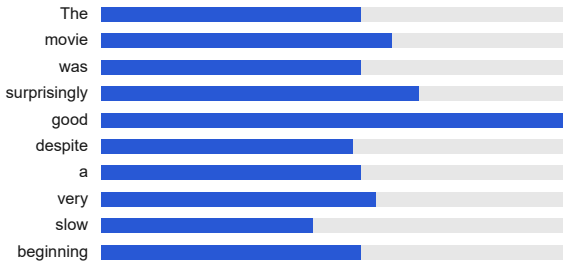
Bài 19 - Experiment 2: Query thay đổi

Cùng sequence, hai query giả lập: Query A - sentiment; Query B - topic/movie.

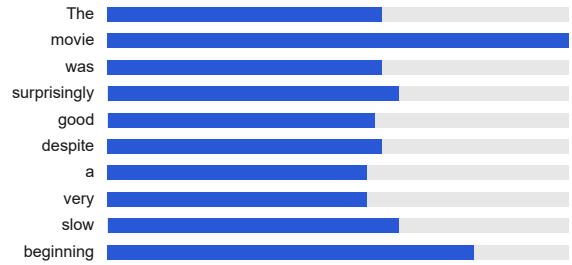
Keys là vector minh họa được thiết kế theo các hướng này, không phải embedding đã huấn luyện.

Token	Score A	Attention A	Score B	Attention B
The	0.000000	0.091111	0.057735	0.090957
movie	0.115470	0.102263	0.577350	0.152933
was	0.000000	0.091111	0.057735	0.090957
surprisingly	0.202073	0.111514	0.115470	0.096363
good	0.577350	0.162297	0.028868	0.088368
despite	-0.028868	0.088518	0.057735	0.090957
a	0.000000	0.091111	0.000000	0.085854
very	0.057735	0.096526	0.000000	0.085854
slow	-0.202073	0.074441	0.115470	0.096363
beginning	0.000000	0.091111	0.346410	0.121396

Query A - sentiment



Query B - topic/movie



Câu hỏi: Attention distribution có thay đổi khi query thay đổi không?

Có. Query A cho trọng số cao nhất ở “good” (0.162297), còn Query B cao nhất ở “movie” (0.152933). Trọng số trung bình thay đổi tuyệt đối 0.021063; tổng trọng số của mỗi query bằng 1. Kết quả minh họa attention distribution phụ thuộc vào query.

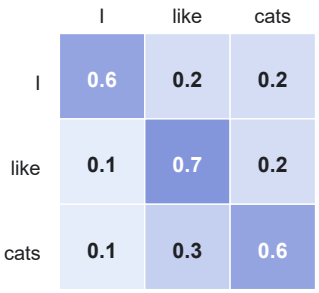
Nguồn output: notebook Google Colab LAB06_Bai19_Query_Changes_Attention, chạy bằng Python 3.

Bài 22 - Attention matrix

Ma trận và heatmap theo dữ liệu ví dụ trong đề bài.

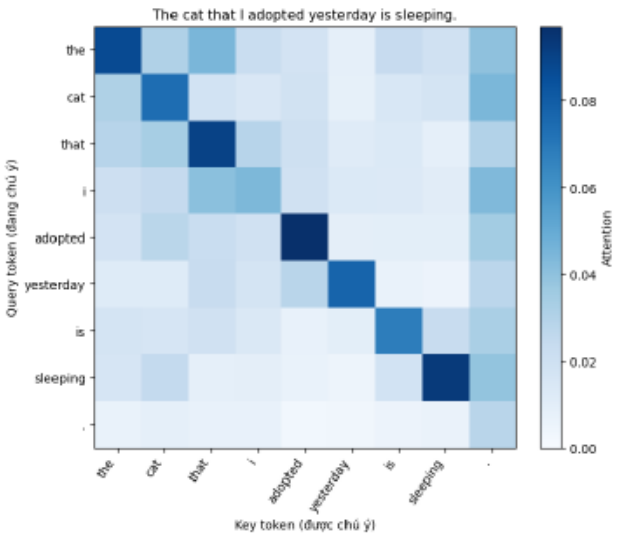
Token	I	like	cats
I	0.6	0.2	0.2
like	0.1	0.7	0.2
cats	0.1	0.3	0.6

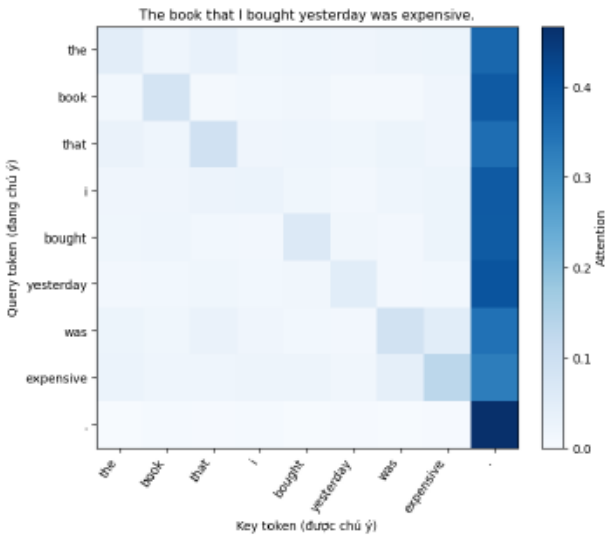
Heatmap



Bài 23 - Attention visualization

Mô hình: bert-base-uncased | Trung bình attention các head ở layer cuối





Token được chú ý nhiều nhất từ ba query token

Query token	Top-5 token được chú ý (attention weight)
cat	. (0.044), the (0.031), adopted (0.018), that (0.018), sleeping (0.017)
adopted	. (0.035), cat (0.028), that (0.023), i (0.019), the (0.018)
sleeping	. (0.039), cat (0.025), is (0.018), the (0.016), i (0.009)

Heatmap thể hiện phân phối attention; không tự nó chứng minh mô hình hiểu ngữ pháp hay ngôn ngữ.

Bài 24 - RNN context và Attention context

IMDb | Vocabulary: 20.000 | Sequence length: 200 | Train/validation/test: 22.500/2.500/25.000 | 3 epochs

Model	Accuracy	F1	Training time (s)	Parameters
LSTM final hidden state	0.7983	0.7656	116.5986	1,313,089
Self-attention context pooling	0.7957	0.7649	169.2912	1,301,058

Câu hỏi cuối bài

LSTM đọc chuỗi tuần tự và dùng final hidden state làm vector cố định, được cập nhật qua từng bước. Self-attention tạo biểu diễn contextualized cho từng token từ thông tin các token khác; attention pooling học trọng số để tổng hợp chúng thành context vector. Do đó LSTM tổng hợp tuần tự vào trạng thái cuối, còn attention giữ biểu diễn theo vị trí trước khi kết hợp có trọng số.

Bài 25 - Sequence length và computational cost

Số phần tử attention matrix được tính bằng T^2; so sánh mức tăng với T = 10.

Sequence length (T)	Attention matrix	Number of elements (T^2)
10	10 x 10	100
100	100 x 100	10,000
500	500 x 500	250,000
1000	1000 x 1000	1,000,000

Growth vs T=10: 1x | 100x | 2,500x | 10,000x

Câu hỏi cuối bài

Self-attention phải tính tương tác giữa từng cặp token, nên attention matrix có T x T phần tử và chi phí bộ nhớ/tính toán tăng theo bình phương độ dài chuỗi, $O(T^2)$. Khi chuỗi dài gấp 10 lần, số phần tử ma trận tăng gấp 100 lần. Vì vậy bộ nhớ và thời gian xử lý trở thành hạn chế đáng kể, đặc biệt với batch lớn và nhiều attention head.

Bài 26 - Application: Context-aware sentiment

IMDb | 15.000 train | 5.000 test | Vocabulary 20.000 | Sequence length 160 | Self-attention 2 heads | 3 epochs

Parameters: 1.300.993 | Training time: 185,9 s | Test loss: 0,5005 | Accuracy: 0,8152 | F1: 0,8285

Sentence	Expected	Prediction	P(pred)	Correct
I like this movie.	positive	positive	0.9867	Yes
I do not like this movie.	negative	negative	0.8570	Yes
I thought this movie would be bad, but it was actually excellent.	positive	positive	0.9890	Yes

Mean attention weight theo token (ba câu đề bài)

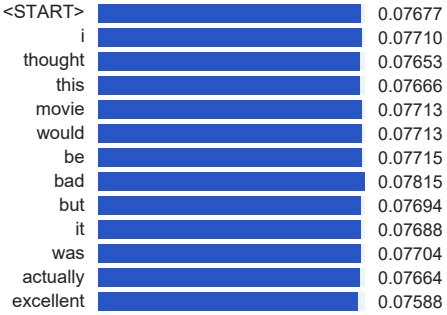
I like this movie.



I do not like this movie.



I thought this movie would be bad, but it was actually excellent.



Một ví dụ đúng và một ví dụ sai từ test set (để tự phân tích)

Đúng | index 1 | Expected: positive | Predicted: positive | p=0.9998
this film requires a lot of patience because it focuses on mood and character development; the plot is very simple and many scenes take place on the same set ...

Sai | index 0 | Expected: negative | Predicted: positive | p=0.9129
please give this one a miss; kristy swanson and the rest of the cast rendered terrible performances; the show is flat flat flat ...

Top mean-attention tokens (held-out correct review)

patience 0.0063 | development 0.0063 | same 0.0063 | frances 0.0063 | interesting 0.0063

Top mean-attention tokens (held-out incorrect review)

terrible 0.0150 | rest 0.0148 | wasn't 0.0148 | give 0.0148 (two positions)

Attention values are model outputs; interpretations are left for the student.

Bài 28 - So sánh LAB 05 và LAB 06

Output Colab: bảng so sánh đặc điểm Attention và recurrent; số liệu thực nghiệm recurrent lấy từ LAB 05, Bài 19.

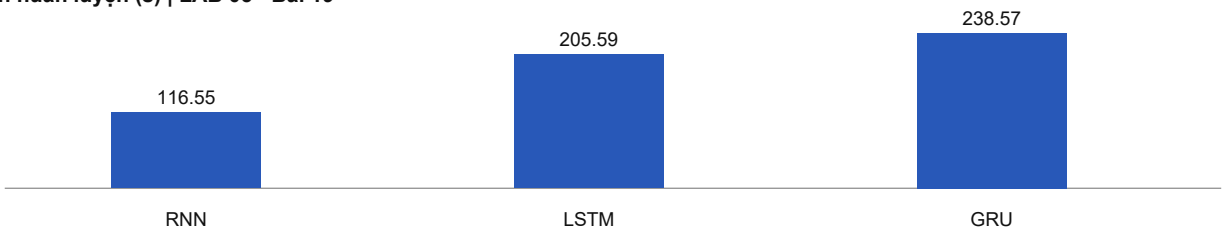
So sánh đặc điểm

Thuộc tính	RNN / LSTM / GRU	Attention
Thứ tự chuỗi	Được xử lý theo thứ tự	Có thể dùng; mỗi token có thể tham chiếu các token khác
Hidden state	Cần trạng thái ẩn để truyền thông tin qua các bước	Không bắt buộc có recurrent hidden state
Tính toán tuần tự	Có; bước sau phụ thuộc bước trước	Không bắt buộc tuần tự theo token trong self-attention
Tương tác trực tiếp giữa token	Gián tiếp qua các bước recurrent	Có thể tính attention giữa các cặp token
Phụ thuộc xa	Khó hơn khi thông tin phải đi qua nhiều bước	Dễ tiếp cận trực tiếp token xa
Tính toán song song	Hạn chế theo chiều thời gian	Có thể song song hóa các token trong một layer
Độ phức tạp theo độ dài T	Gần O(T) bước recurrent (chưa tính kích thước ẩn)	O(T²) cho full self-attention theo số cặp token
Biểu diễn ngữ cảnh	Trạng thái recurrent tổng hợp thông tin đã đọc	Tổng có trọng số của các token được attention

Số liệu recurrent từ LAB 05 - Bài 19

Model	Parameters	Training time (s)	Accuracy	F1
RNN	1,288,321	116.55	0.7946	0.7901
LSTM	1,313,089	205.59	0.8109	0.8293
GRU	1,305,025	238.57	0.8107	0.8320

Thời gian huấn luyện (s) | LAB 05 - Bài 19



Câu hỏi cuối bài

1. Attention giải quyết vấn đề gì của recurrent processing?
Attention cho phép mỗi token truy cập trực tiếp thông tin từ các token khác, giảm việc phải truyền thông tin qua nhiều bước recurrent. Các token trong một layer cũng có thể được tính song song.

2. Attention tạo ra vấn đề mới nào?
Full self-attention có chi phí bộ nhớ và tính toán O(T²) theo độ dài chuỗi. Attention thuần cũng không biểu diễn thứ tự token nếu không bổ sung positional information.

Bài 29 - Attention không miễn phí

Sequence length và kích thước attention matrix

Sequence length (T)	Attention matrix	Elements (T²)	FP32 memory (decimal MB)
1,000	1,000 × 1,000	1,000,000	4
10,000	10,000 × 10,000	100,000,000	400

T tăng từ 1,000 lên 10,000 (gấp 10); số phần tử ma trận tăng gấp 100 lần, theo quy luật T².
Bộ nhớ FP32 trên chỉ tính một ma trận cho một mẫu và một head.

Quadratic growth of full self-attention

